

Wireshark twoo twooo two twoo...

 **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

Can you find the flag?

Tools Used

- `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read PCAP metadata
 - **Wireshark** – GUI tool for analyzing network traffic
 - `tshark` – Command-line Wireshark tool for packet analysis
 - `awk` – Text processing and field extraction
 - `cut` – Extract specific fields from output
 - `uniq` – Remove duplicate lines
 - `tr` – Delete newlines and carriage returns
 - `base64` – Decode Base64-encoded content
-

Complete Solution

Step 1: Download and Prepare

Download the PCAP file named `shark2.pcapng` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type:

```
file shark2.pcapng
```

Expected output:

```
shark2.pcapng: pcapng capture file - version 1.0
```

The file is a **PCAPNG** (PCAP Next Generation) capture file containing network traffic.

Step 3: Search for Visible Text

Check if the flag appears as plain text:

```
strings shark2.pcapng | grep -i "picoCTF"
```

Output (excerpt):

```
picoCTF{bfe48e8500c454d647c55a4471985e776a07b26cba64526713f43758599aa98b}  
picoCTF{bda69bdf8f570a9aaab0e4108a0fa5f64cb26ba7d2269bb63f68af5d98b98245}  
picoCTF{fe83bcb6cfd43d3b79392f6a4232685f6ed4e7a789c2ce559cf3c1ab6adbe34b}  
picoCTF{711d3893d90f100c15e10ef4842abeed3a830f8237c1257cd47389646da97810}  
picoCTF{3cf1e22d489fcfb6bb312a34f46c8699989ed043406134331452d11ce73cd59e}  
...
```

Many fake flags appear! The real flag is hidden elsewhere.

Step 4: Examine Metadata

```
exiftool shark2.pcapng
```

Output:

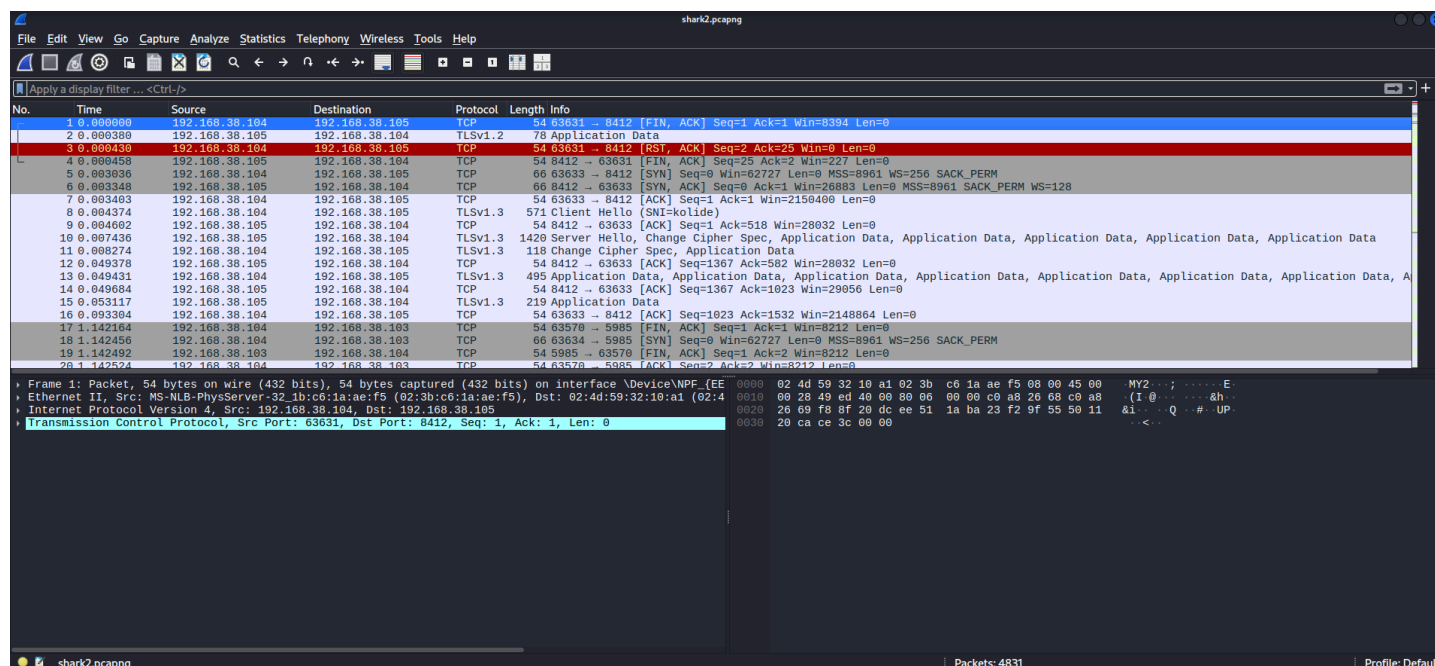
```
ExifTool Version Number      : 13.50  
File Name                    : shark2.pcapng  
Directory                    : .
```

File Size : 3.5 MB
File Modification Date/Time : 2026:05:18 17:53:17+02:00
File Access Date/Time : 2026:05:18 17:53:37+02:00
File Inode Change Date/Time : 2026:05:18 17:53:25+02:00
File Permissions : -rw-rw-rw-
File Type : PCAPNG
File Type Extension : pcapng
MIME Type : application/vnd.tcpdump.pcap
PCAP Version : PCAPNG 1.0
Byte Order : Little-endian (Intel, II)
Hardware : Intel(R) Xeon(R) CPU E5-2676 v3 @ 2.40GHz (with SSE4.2)
User Application : Dumpcap (Wireshark) 3.2.5 (v3.2.5-0-ged20ddea8138)
Link Type : IEEE 802.3 Ethernet
Device Name : \Device\NPF_{EE54A6A6-8BD8-498A-995A-DE0B78012FBE}
Description : Ethernet 4
Time Stamp Resolution : 1e-06
Filter : 0: not tcp port 3389
Operating System : 64-bit Windows 10 (1909), build 18363
Time Stamp : 2020:08:10 03:37:39.475628+02:00

The metadata doesn't reveal the flag.

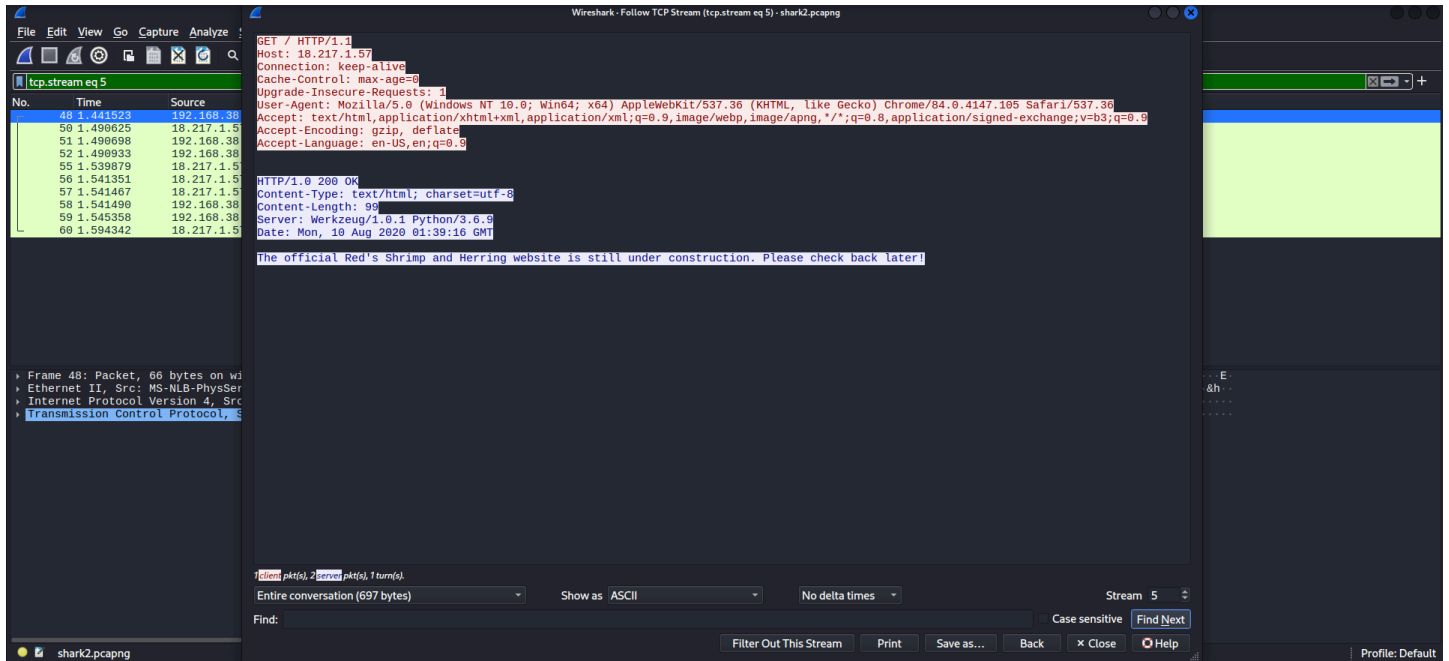
Step 5: Analyze with Wireshark

Open the file in Wireshark:

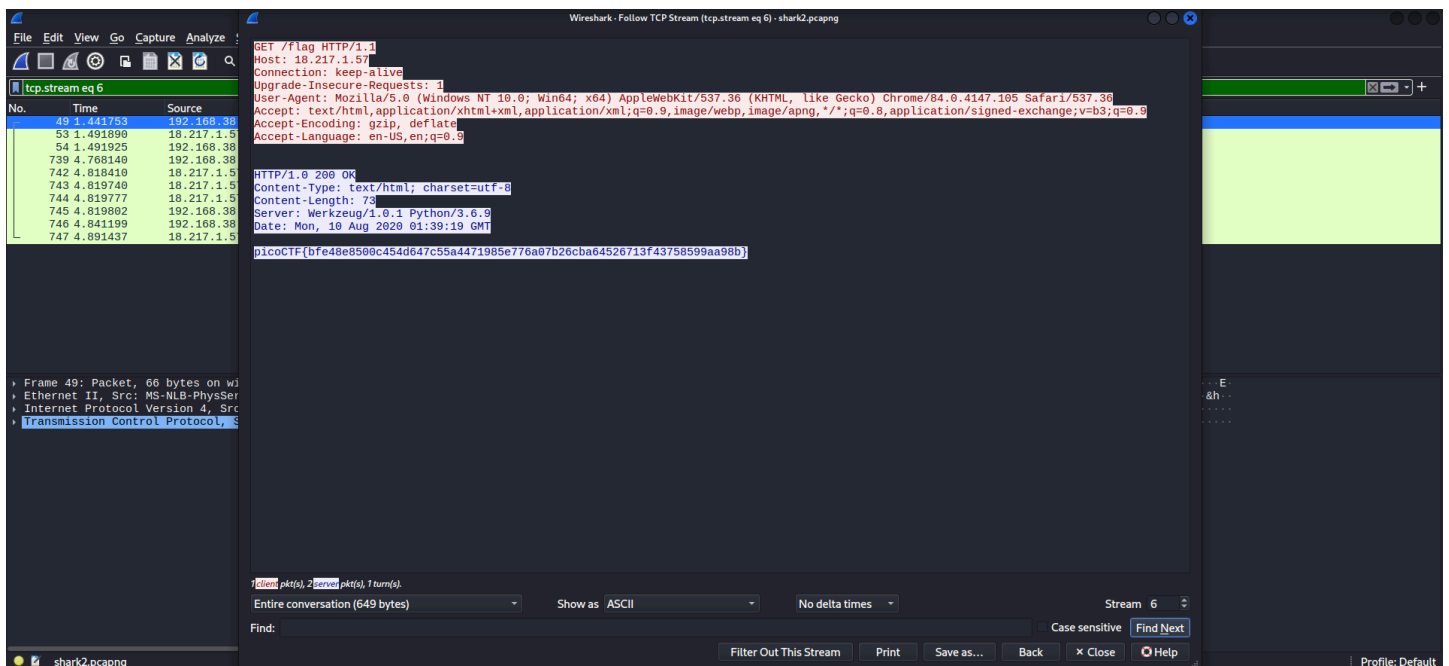


Two interesting TCP streams catch our attention:

- **tcp.stream eq 5** – Contains a message: “The official Red’s Shrimp and Herring website is still under construction. Please check back later!”



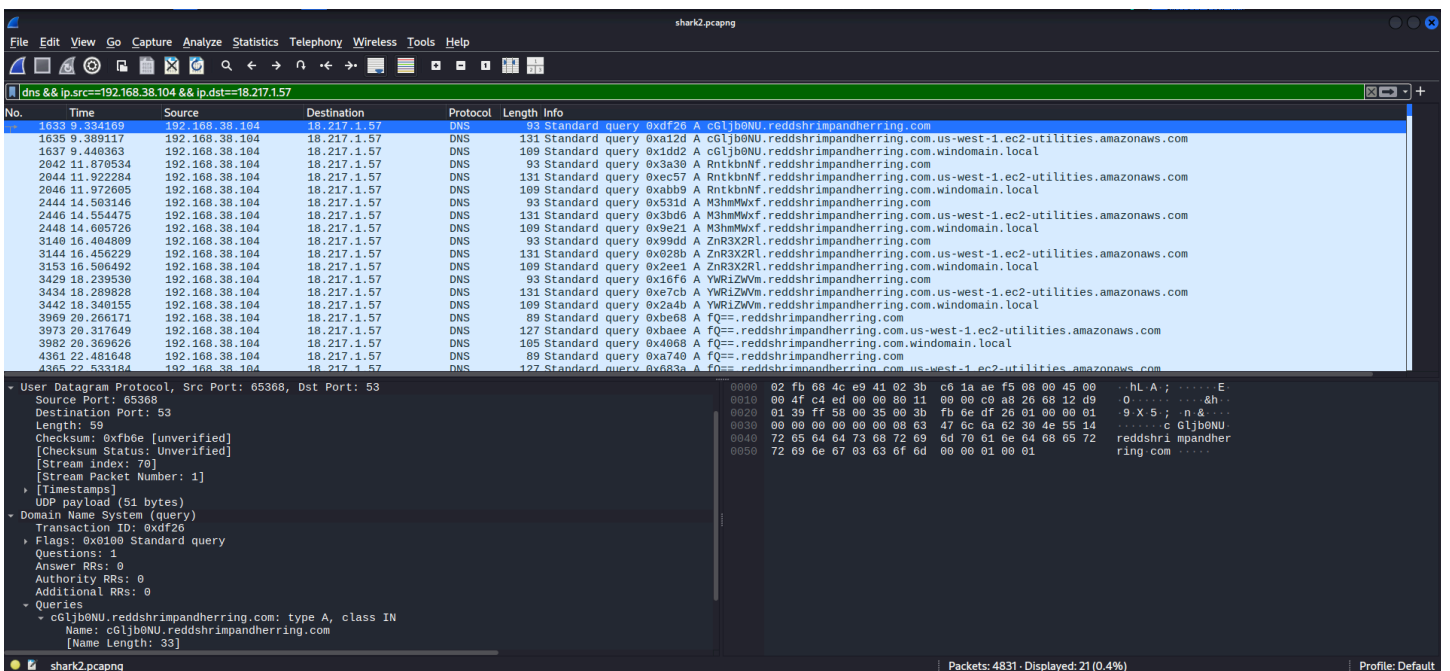
- **tcp.stream eq 6** – Contains a fake flag:
`picoCTF{bfe48e8500c454d647c55a4471985e776a07b26cba64526713f43758599aa98b}`



Since the official website isn’t working, we need to look for subdomains in **DNS queries**.

Step 6: Examine DNS Traffic

Look for DNS queries from the client (192.168.38.104) to an external DNS server (18.217.1.57):



Notice that the DNS queries contain **Base64-encoded subdomains**!

Step 7: Extract DNS Queries with tshark

First, filter for DNS queries from the client to the external DNS server:

```
tshark -nr shark2.pcapng -Y 'dns && ip.src==192.168.38.104 && ip.dst==18.217.1.57'
```

Output:

```
1633  9.334169 192.168.38.104 → 18.217.1.57  DNS 93 Standard query 0xdf26 A
cGljb0NU.reddshrimpandherring.com
1635  9.389117 192.168.38.104 → 18.217.1.57  DNS 131 Standard query 0xa12d A
cGljb0NU.reddshrimpandherring.com.us-west-1.ec2-utilities.amazonaws.com
1637  9.440363 192.168.38.104 → 18.217.1.57  DNS 109 Standard query 0x1dd2 A
cGljb0NU.reddshrimpandherring.com.windomain.local
2042  11.870534 192.168.38.104 → 18.217.1.57  DNS 93 Standard query 0x3a30 A
RntkbnNf.reddshrimpandherring.com
2044  11.922284 192.168.38.104 → 18.217.1.57  DNS 131 Standard query 0xec57 A
RntkbnNf.reddshrimpandherring.com.us-west-1.ec2-utilities.amazonaws.com
2046  11.972605 192.168.38.104 → 18.217.1.57  DNS 109 Standard query 0xab9 A
RntkbnNf.reddshrimpandherring.com.windomain.local
2444  14.503146 192.168.38.104 → 18.217.1.57  DNS 93 Standard query 0x531d A
M3hmMWxf.reddshrimpandherring.com
2446  14.554475 192.168.38.104 → 18.217.1.57  DNS 131 Standard query 0x3bd6 A
M3hmMWxf.reddshrimpandherring.com.us-west-1.ec2-utilities.amazonaws.com
2448  14.605726 192.168.38.104 → 18.217.1.57  DNS 109 Standard query 0x9e21 A
M3hmMWxf.reddshrimpandherring.com.windomain.local
3140  16.404809 192.168.38.104 → 18.217.1.57  DNS 93 Standard query 0x99dd A
ZnR3X2Rl.reddshrimpandherring.com
3144  16.456229 192.168.38.104 → 18.217.1.57  DNS 131 Standard query 0x028b A
ZnR3X2Rl.reddshrimpandherring.com.us-west-1.ec2-utilities.amazonaws.com
3153  16.506492 192.168.38.104 → 18.217.1.57  DNS 109 Standard query 0x2ee1 A
ZnR3X2Rl.reddshrimpandherring.com.windomain.local
3429  18.239530 192.168.38.104 → 18.217.1.57  DNS 93 Standard query 0x16f6 A
YWRiZWVm.reddshrimpandherring.com
3434  18.289828 192.168.38.104 → 18.217.1.57  DNS 131 Standard query 0xe7cb A
YWRiZWVm.reddshrimpandherring.com.us-west-1.ec2-utilities.amazonaws.com
3442  18.340155 192.168.38.104 → 18.217.1.57  DNS 109 Standard query 0x2a4b A
YWRiZWVm.reddshrimpandherring.com.windomain.local
3969  20.266171 192.168.38.104 → 18.217.1.57  DNS 89 Standard query 0xbe68 A
fQ==.reddshrimpandherring.com
```

```
3973  20.317649 192.168.38.104 → 18.217.1.57  DNS 127 Standard query 0xbaee A
fQ==.reddshrimpandherring.com.us-west-1.ec2-utilities.amazonaws.com
3982  20.369626 192.168.38.104 → 18.217.1.57  DNS 105 Standard query 0x4068 A
fQ==.reddshrimpandherring.com.windomain.local
4361  22.481648 192.168.38.104 → 18.217.1.57  DNS 89 Standard query 0xa740 A
fQ==.reddshrimpandherring.com
4365  22.533184 192.168.38.104 → 18.217.1.57  DNS 127 Standard query 0x683a A
fQ==.reddshrimpandherring.com.us-west-1.ec2-utilities.amazonaws.com
4374  22.583745 192.168.38.104 → 18.217.1.57  DNS 105 Standard query 0x7418 A
fQ==.reddshrimpandherring.com.windomain.local
```

The subdomain parts (`cGljb0NU` , `RntkbnNf` , etc.) look like **Base64**!

Step 8: Filter Unique Subdomains

We only need the queries ending with `.windomain.local` (these contain the Base64 data without duplicates):

```
tshark -nr shark2.pcapng -Y 'dns && ip.src==192.168.38.104 && ip.dst==18.217.1.57 &&
frame contains "local"'
```

Output:

```
1637   9.440363 192.168.38.104 → 18.217.1.57  DNS 109 Standard query 0x1dd2 A
cGljb0NU.reddshrimpandherring.com.windomain.local
2046  11.972605 192.168.38.104 → 18.217.1.57  DNS 109 Standard query 0xabbb9 A
RntkbnNf.reddshrimpandherring.com.windomain.local
2448  14.605726 192.168.38.104 → 18.217.1.57  DNS 109 Standard query 0x9e21 A
M3hmMWxf.reddshrimpandherring.com.windomain.local
3153  16.506492 192.168.38.104 → 18.217.1.57  DNS 109 Standard query 0x2ee1 A
ZnR3X2Rl.reddshrimpandherring.com.windomain.local
3442  18.340155 192.168.38.104 → 18.217.1.57  DNS 109 Standard query 0x2a4b A
YWRiZWVm.reddshrimpandherring.com.windomain.local
3982  20.369626 192.168.38.104 → 18.217.1.57  DNS 105 Standard query 0x4068 A
fQ==.reddshrimpandherring.com.windomain.local
4374  22.583745 192.168.38.104 → 18.217.1.57  DNS 105 Standard query 0x7418 A
fQ==.reddshrimpandherring.com.windomain.local
```

Step 9: Extract Just the Subdomain Parts

Extract the 12th field (the subdomain part before the first dot):

```
tshark -nr shark2.pcapng -Y 'dns && ip.src==192.168.38.104 && ip.dst==18.217.1.57 && frame contains "local"' | awk '{print $12}' | cut -d '.' -f1
```

Output:

```
cG1jb0NU
RntkbnNf
M3hmMWxf
ZnR3X2Rl
YW RiZWVm
fQ==
fQ==
```

Step 10: Remove Duplicates and Decode Base64

Use `uniq` to remove duplicates, then decode the concatenated Base64:

```
tshark -nr shark2.pcapng -Y 'dns && ip.src==192.168.38.104 && ip.dst==18.217.1.57 && frame contains "local"' | awk '{print $12}' | cut -d '.' -f1 | uniq | tr -d '\r\n' | base64 -d
```

Output:

```
picoCTF{<REDACTED>}
```

One-Liner Solution

Here's the complete pipeline to extract and decode the flag in one command:

```
tshark -nr shark2.pcapng -Y 'dns && ip.src==192.168.38.104 && ip.dst==18.217.1.57 && frame contains "local"' | awk '{print $12}' | cut -d '.' -f1 | uniq | tr -d '\r\n' | base64 -d
```

Final Result

picoCTF{<REDACTED>}

Note: The actual flag is redacted here. In the real challenge, running the commands above will reveal the complete flag.

Key Concepts

Concept	Description
PCAPNG	Next-generation PCAP format that supports multiple interfaces and enhanced metadata.
DNS Tunneling	Hiding data in DNS queries (subdomains) to bypass firewalls or exfiltrate information.
Base64 Encoding	A common encoding used to represent binary data in ASCII text (detected by = padding).
Subdomain Data Exfiltration	Data can be split across multiple DNS queries, each using a subdomain label.
tshark	Command-line version of Wireshark for automated packet analysis.
awk / cut	Text processing tools for extracting specific fields.
uniq	Removes duplicate lines from sorted output.

Pro Tips

- Look for unusual DNS queries** – Long subdomains or repeated queries often indicate DNS tunneling.

2. **Base64 can be detected by = padding** – The presence of `=` characters in subdomains is a dead giveaway.
3. **Check DNS traffic** – When HTTP/HTTPS traffic doesn't reveal the flag, DNS is the next place to look.
4. **Use `frame contains` filters** – `frame contains "local"` filters packets containing specific text.
5. **Concatenate Base64 parts** – Data split across multiple DNS queries needs to be reassembled before decoding.
6. **`uniq` removes duplicates** – Some queries repeat; `uniq` ensures each part is included only once.

Alternative Base64 Decoding Tools

Tool	Command / Link
Linux <code>base64</code>	<code>echo "cGljb0NU..." base64 -d</code>
CyberChef	https://gchq.github.io/CyberChef/ (Operation: From Base64)
Python	<code>import base64; base64.b64decode("cGljb0NU...")</code>
Online	https://www.base64decode.org/